

DAT12-1

Introduction to Data with Spreadsheet

Observing, organising, and describing data

Look at the data before you touch it.

Albert School — 16 hours

Preface

This is, for most of you, the first course of your data education — and it is on purpose the most humble one. Before you train a model, query a database, or write a line of Python, you will spend the entire course in a tool you already half know: the spreadsheet. No prior knowledge is assumed. What **is** assumed is a willingness to slow down.

The central habit this course builds is captured in its tagline: **look at the data before you touch it**. Beginners reach immediately for a formula, a chart, an average. Experienced data people do something that looks like doing nothing — they scroll, they sort, they squint at a column, they ask “does this even make sense?” — before computing anything. They know that a number computed from dirty data is worse than no number, because it carries false confidence; and that a chart that answers the wrong question misleads exactly as much as a wrong calculation.

A spreadsheet is the perfect place to learn this, because everything is visible. There is no hidden pipeline. Every value sits in a cell you can click on; every mistake — a number stored as text, a duplicated row, a misleading axis — is right there in front of you, **if you have trained your eye to see it**. That training is the real subject of this course. By the end you will be able to take a genuine business question, answer it using only spreadsheet operations, and document your work so that a colleague can reproduce it without asking you a single question.

Contents

Preface	2
1 The Spreadsheet Environment	3
1.1 A grid that means something	3
1.2 Values, types, and the difference formatting hides	3
2 Core Functions	4
2.1 A formula is a question asked of the data	4
2.2 Summarising a column: SUM and AVERAGE	4
2.3 Branching on a condition: IF	5
2.4 Counting what matches: COUNTIF	5
2.5 Looking up a related value: VLOOKUP and INDEX/MATCH	5
3 Sorting, Filtering, and Grouping	6
3.1 Three ways to make a pattern visible	6
4 Pivot Tables	7
4.1 Grouping, made interactive	7
5 Data Quality at the Spreadsheet Level	8
5.1 The errors you can see if you look	8
6 Visualization	8
6.1 The right chart answers the question; the wrong one lies	8
7 From Business Question to Spreadsheet Answer	10
7.1 The whole course, in three steps	10
8 Documentation and Reproducibility	10
8.1 An analysis no one can reproduce is not finished	10

1 The Spreadsheet Environment

1.1 A grid that means something

Open any spreadsheet and you see a grid: columns labelled with letters (A, B, C, ...) and rows numbered (1, 2, 3, ...). Each rectangle is a **cell**, named by its column and row — **B4** is the cell in column B, row 4. This addressing is the whole reason spreadsheets work: every value has a name, so every formula can refer to it.

Almost always, good data follows one quiet convention that everything else in this course depends on:

Definition 1 (Tidy tabular layout) Data is laid out as a **table** in which each **column** is one variable (one kind of thing measured), each **row** is one record (one observation), and the first row holds the **column headers** naming the variables. One cell holds one value.

Worked example — Reading a small sales table. A shop records its sales like this:

Date	Product	Units	Price (€)
2026-01-04	Coffee	12	3.50
2026-01-04	Tea	7	2.80
2026-01-05	Coffee	15	3.50

Each column is one variable (a date, a product name, a count, a price); each row is one sale. Because the layout is regular, you can already **read** it before computing anything: coffee outsells tea, and sells for more. That reading — done by eye, before any formula — is the reflex the course is built around.

1.2 Values, types, and the difference formatting hides

Click a single cell and the spreadsheet must decide what kind of thing it holds. This decision — the cell's **type** — is invisible but decisive.

Definition 2 (Type vs formatting) A cell's **type** is how the spreadsheet interprets its content — a **number**, a **date**, or **text**. **Formatting** controls only how that content is **displayed** (decimal places, a currency symbol, a date style) and never changes the stored value. Two cells can look identical on screen and yet be of different types.

This gap between what you see and what is stored is the source of the most common beginner bug in all of spreadsheet work, so we meet it immediately.

Worked example — The number that will not add up. You type 1 200 into one cell (with a space, as you might write it by hand) and 1200 into another. On screen they look almost the same. But 1 200 contains a space, so the spreadsheet stores it as **text**; 1200 is stored as a **number**. Ask for their sum and you get 1200, not 2400 — the text cell is silently treated as zero. Nothing turned red. Nothing warned you. The total is simply wrong, and looks fine.

Common pitfall A number stored as text will not be summed or averaged correctly, and a date stored as text will not sort chronologically — “10/01” may sort before “9/01” because

they are compared letter by letter, not as dates. Always check the **stored type**, not just the displayed value. A reliable tell: numbers and dates usually align to the **right** of a cell by default, text to the **left**. A column of “numbers” that hugs the left edge is a column of text in disguise.

	A	B	C
1	Product	Units	Price
2	Coffee	12	3.50
3	Tea	7	2.80
4	Cocoa	9	4.00



left-aligned → stored as *text*, not a number

Figure 1: The cell reference B2 names column B, row 2. Numbers align right by default; the 9 in the Units column is left-aligned, betraying that it was stored as text and will be ignored by SUM.

Remark Correct entry, then, is mostly discipline: one kind of value per column, numbers stored as numbers, dates stored as dates, and text reserved for genuine labels. Editing a cell amends its content; formatting changes only its appearance. Get the **types** right at entry and every later operation — sums, sorting, charts — behaves. Get them wrong and every later operation inherits the error.

2 Core Functions

2.1 A formula is a question asked of the data

A **formula** begins with = and computes a result from values or, more usefully, from **cell references**. Writing =B2*C2 next to our sales table multiplies units by price to give a line total — and because it refers to cells rather than to the numbers themselves, correcting a value in B2 updates the total automatically.

The skill in this chapter is not memorising functions; it is knowing **which** function a given question calls for. We meet five, each the right tool for a different shape of question.

2.2 Summarising a column: SUM and AVERAGE

Definition 3 (SUM and AVERAGE) SUM(range) adds the numbers in a range of cells. AVERAGE(range) returns their arithmetic mean — the sum divided by how many numbers there are. A **range** like C2:C100 names every cell from C2 down to C100.

Worked example — Total and typical revenue. With line totals in column D, =SUM(D2:D100) answers “how much did we take in altogether?” and =AVERAGE(D2:D100) answers “what was a typical sale worth?” The two questions are different, and so are the functions: a total and a typical value are not interchangeable. Reporting the sum when someone asked for the average — or vice versa — is a wrong answer even though every number is correct.

Common pitfall AVERAGE ignores genuinely empty cells but treats a cell containing 0 as a real value. If a missing measurement was recorded as 0, the average is dragged down by data that should not be there at all. This is the first place where the data-quality habits of Chapter 5 reach back and change a result — **which** blanks mean “zero” and which mean “unknown” is a decision, not a detail.

2.3 Branching on a condition: IF

Definition 4 (IF) IF(condition, value-if-true, value-if-false) evaluates a condition and returns the first value when it holds and the second when it does not. For example =IF(D2>50, "large", "small") labels each sale.

Worked example — Flagging sales that need attention. A manager wants every order above €100 marked for a follow-up call. =IF(D2>100, "call", "") writes “call” beside the big orders and leaves the rest blank. The condition encodes the business rule; the two outcomes encode what to do about it. IF is how a human decision becomes a column.

2.4 Counting what matches: COUNTIF

Definition 5 (COUNTIF) COUNTIF(range, criterion) counts how many cells in a range meet a criterion. The criterion can be a value ("Coffee"), or a comparison written in quotes (">100").

Worked example — How many, not how much. =COUNTIF(A2:A100, "Coffee") counts how many sales were of coffee; =COUNTIF(D2:D100, ">100") counts how many orders exceeded €100. Note the difference from SUM: COUNTIF answers “how **many** rows?”, SUM answers “how much in **total**?”. Confusing a count with a sum is among the most common spreadsheet errors — “we had 40 big orders” is a very different statement from “big orders were worth €40.”

2.5 Looking up a related value: VLOOKUP and INDEX/MATCH

Often the value you need lives in **another** table. A sales sheet lists a product **code**; the price for each code sits in a separate price list. A **lookup** fetches the matching value across tables.

Definition 6 (VLOOKUP and INDEX/MATCH) VLOOKUP(key, table, column, FALSE) searches for key in the **first** column of table and returns the value from the numbered column of the matching row (the final FALSE demands an **exact** match).

INDEX(range, row, column) returns the value at a given position; MATCH(key, range, 0) returns the **position** of key within a range. Nested, =INDEX(prices, MATCH(code, codes, 0), 1) performs a lookup with neither of VLOOKUP’s restrictions: the key need not sit in the first column, and the returned value may lie to its **left**.

Price list		
code	price	
C01	3.50	Sale has code T01. Find its price: <code>VLOOKUP("T01", table, 2, FALSE)</code> returns 2.80.
T01	2.80	
K01	4.00	

Figure 2: A lookup matches a key (T01) in the first column of the price list and returns the value beside it. `VLOOKUP` requires the key in the first column and reads to the right; `INDEX/MATCH` lifts both restrictions.

Common pitfall `VLOOKUP` silently fails in two classic ways. First, omitting the final `FALSE` lets it return an **approximate** match — a different product’s price — without any error. Second, it can only return a column to the **right** of the key; if the value you need sits to the left, `VLOOKUP` cannot reach it and beginners rearrange their data to suit the tool. Prefer `INDEX/MATCH`, which has neither trap.

Chapter takeaways. A formula refers to cells, so results update when data changes. Match the function to the question: `SUM` for a total, `AVERAGE` for a typical value, `IF` to turn a rule into a column, `COUNTIF` for “how many”, and a lookup (`VLOOKUP` or, more safely, `INDEX/MATCH`) to pull a related value from another table.

3 Sorting, Filtering, and Grouping

3.1 Three ways to make a pattern visible

A thousand rows hide their patterns from the eye. Three operations — sorting, filtering, grouping — rearrange or condense the table until the pattern or the anomaly stands out. None of them computes anything new; they only change **what you see**, which is exactly the point of looking before touching.

Definition 7 (Sort, filter, group) **Sorting** reorders the rows by the values in one or more columns. **Filtering** hides the rows that fail a stated condition, leaving the rest in view (the data is not deleted, only concealed). **Grouping** collapses rows that share a value in some column and reports a summary — a count, sum, or average — for each group.

Worked example — The same table, three questions. Take a year of sales.

- **Sort** by revenue, descending: the biggest orders jump to the top, and an impossible €99,999 sale — a typo — surfaces at the very top where you cannot miss it.
- **Filter** to Product = “Tea”: only tea sales remain, isolating the question “how is tea doing?” from the noise of everything else.
- **Group** by month and sum revenue: twelve numbers replace ten thousand rows, and the summer slump becomes obvious at a glance.

Each operation answered a different question without changing a single value.

Common pitfall When you sort, sort the **whole table**, not one column. Sorting a single column while its neighbours stay put scrambles every row — prices end up beside the wrong products — and, like the text-number trap, it produces a sheet that looks orderly while being silently corrupted. If in doubt, check that one familiar row still holds together after the sort.

4 Pivot Tables

4.1 Grouping, made interactive

Grouping by hand is fine for one question. But business questions multiply — revenue by month, then by month **and** product, then average rather than total — and redoing the grouping each time is slow and error-prone. A **pivot table** is a machine for grouping that you steer by dragging fields.

Definition 8 (Pivot table) A **pivot table** summarises a source table by grouping rows according to one or more fields and computing an **aggregate** (count, sum, average, ...) of another field for each group. The chosen **row fields** and **column fields** become the margins of the result; each **cell** of the body holds the aggregate for that combination of row and column.

column field: month

	Σ revenue	Jan	Feb
row field: product	Coffee	1 260	980
	Tea	392	560

Figure 3: A pivot table with product as the row field, month as the column field, and summed revenue in the body. Each body cell answers “how much revenue, for this product, in this month?” — one grouping, read four ways.

Worked example — From pivot cell to business sentence. The pivot above shows Coffee/January = €1 260. Building the table is mechanical; the value is in the **reading**. “Coffee earned €1 260 in January, more than three times tea’s €392 — coffee carries our winter revenue” is a business sentence a manager can act on. A pivot table that is built but not **interpreted** is just a smaller spreadsheet.

Common pitfall A pivot table inherits every flaw of its source. If “Coffee” and “coffee” (with a trailing space) both appear, the pivot lists them as **two** products and splits the revenue between them — a wrong answer produced by a correct machine. Clean the data (Chapter 5) **before** you trust a pivot.

5 Data Quality at the Spreadsheet Level

5.1 The errors you can see if you look

The course's tagline pays off here. Most data problems that wreck an analysis are visible at the spreadsheet level — if you inspect the data before computing. Three kinds account for the great majority.

Definition 9 (Three common data-quality issues)

- **Duplicates:** rows that repeat the same record, inflating every count and sum.
- **Inconsistent formats:** the same value entered different ways — "FR" vs "France", 01/02/2026 vs 2026-02-01, Coffee vs coffee — so that equal things are not recognised as equal.
- **Missing values:** cells left blank where a value was expected, which downstream functions may treat as zero, skip, or choke on.

Worked example — One dirty column, three corrections. A “Country” column contains France, france, FR, Frace, and three blanks.

- **Sort** the column: the variants cluster together and the typo Frace sits beside France, easy to spot and fix.
- **Standardise:** decide on one spelling (France) and replace the variants, so a later COUNTIF(., "France") actually counts them all.
- **Missing values:** for each blank, **decide** — is the country genuinely unknown (leave blank and exclude), or obviously inferable from another column (fill it)? Record the decision; it is part of the analysis, not a tidy-up.

Common pitfall Never silently delete a row or overwrite a value while “cleaning”. Each correction is a decision that changes the result and must be **recorded** (Chapter 8), because someone reproducing your work needs to know not just the final numbers but every judgement that produced them. Cleaning without a record is how an analysis becomes impossible to reproduce — and impossible to defend.

Remark The tools of cleaning are the tools you already have: **sorting** brings duplicates and variants together, **filtering** isolates blanks (**is empty**) or suspicious values, and COUNTIF quantifies how widespread a problem is before you start fixing it. Data quality is not a separate skill; it is the earlier operations turned on the data itself.

6 Visualization

6.1 The right chart answers the question; the wrong one lies

A chart is an argument made of ink. Choosing the chart type is choosing what question you are answering, and three types cover most early data work — each matched to a different shape of question.

Definition 10 (Three chart types, three questions)

- A **bar chart** compares a quantity **across categories** (revenue by product).

- A **line chart** shows how a quantity **changes along an ordered axis**, usually time (revenue by month).
- A **scatter plot** shows the **relationship between two numeric variables**, one on each axis (advertising spend vs sales).

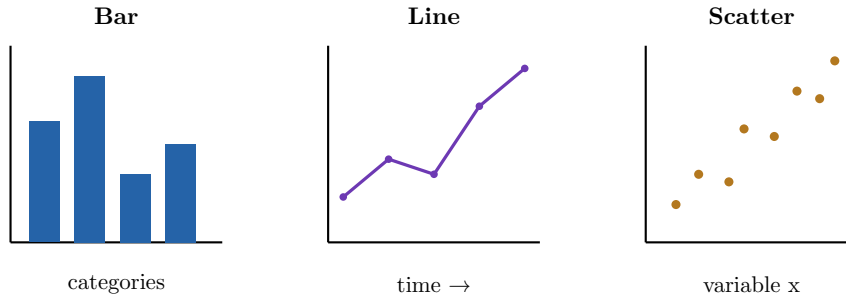


Figure 4: Each chart type fits a different question. Bars compare across categories; the line follows a quantity over time; the scatter reveals whether two variables move together. Using one where another belongs misstates the question.

Worked example — Matching chart to question.

- “Which product sells most?” — a comparison across categories → **bar chart**.
- “Are sales growing?” — change over an ordered time axis → **line chart**.
- “Does spending more on ads bring more sales?” — relationship between two numbers → **scatter plot**.

Put months on a bar chart and you lose the sense of a trend; put unordered products on a line chart and the connecting line implies a progression that does not exist. The chart type is not decoration — it is part of the claim.

Common pitfall A misleading chart is as dangerous as a wrong model. The classic deception is a bar chart whose vertical axis starts not at zero but at, say, 95: a rise from 96 to 99 then **looks** like a tripling. The data is honest; the axis lies. When you read a chart, check the axes first; when you make one, start bar-chart axes at zero unless you have a stated reason not to.

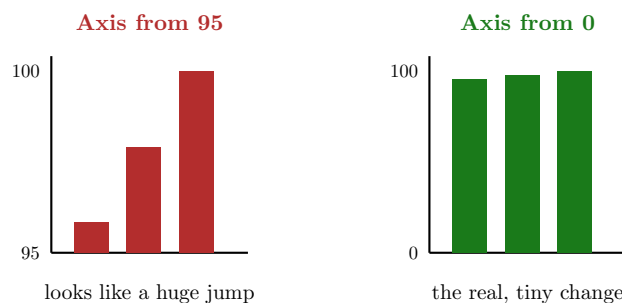


Figure 5: The same three values (96, 98, 100) on a truncated axis (left) and an honest zero-based axis (right). The data is identical; only the left chart lies.

7 From Business Question to Spreadsheet Answer

7.1 The whole course, in three steps

Everything so far — entry, functions, sorting, pivots, cleaning, charts — are means to one end: answering a real question someone actually has. Doing that well is a disciplined three-step move.

Definition 11 (Question → operations → interpretation)

1. **Formulate** the question precisely enough that you would recognise the answer if you saw it (“which product had the highest average order value in Q1?”, not “how are products doing?”).
2. **Select** the operations that answer exactly that — the functions, sorting, filtering, grouping, pivot, or chart the question requires, and no more.
3. **Interpret** the result back in the question’s own terms, stating what it means rather than reporting a bare number.

Worked example — A question answered end to end. **Question:** “Did our January promotion lift coffee sales?”

- **Formulate:** compare coffee revenue in the promotion week against the surrounding weeks.
- **Select:** filter to **Product = Coffee**; group revenue by week (a pivot); inspect with a line chart, ordered by week.
- **Interpret:** “Coffee revenue rose 40% in the promotion week, then fell back to its prior level — the promotion shifted timing more than it added sales.”

Notice what we did **not** do: no scatter plot, no average across all products, no extra cleverness. The discipline is to use only the operations the question needs and to end with a sentence, not a number.

Common pitfall The commonest failure is producing numbers without an answer — a dashboard of sums and charts that never returns to the question that was asked. A correct total that does not answer the question is not a partial answer; it is no answer. Always close the loop back to the words of the question.

8 Documentation and Reproducibility

8.1 An analysis no one can reproduce is not finished

The final discipline is the one that separates a private spreadsheet from a piece of professional work: someone else — or you, six months later — must be able to follow it and arrive at the same result **without asking you anything**.

Definition 12 (Reproducible analysis) A spreadsheet analysis is **reproducible** when another person can follow it, understand each step, and reach the same result without assistance. This requires recording the **source** of the data, the **steps** taken in order, the **assumptions** and judgements made (especially how data-quality issues were handled), and **where each reported result comes from**.

Worked example — The difference a note makes. Two analysts both report “average order value: €38.” The first hands over a sheet of formulas. The second adds a short notes tab: “Source: 2026 sales export, downloaded 4 Feb. Removed 12 duplicate rows (see column K flag). Excluded 3 orders with missing prices. Average computed over the remaining 2 940 orders in D2:D2941.” Only the second analysis can be **trusted**, because only the second can be **checked**. The number is the same; the work is not.

Common pitfall Undocumented cleaning is the silent killer of reproducibility. If you removed duplicates, standardised formats, or excluded blanks (Chapter 5) without writing it down, your final numbers cannot be reconstructed from the raw data — and an analysis that cannot be reconstructed cannot be defended when someone questions it. Document the judgements, not just the formulas.

Course takeaways. A spreadsheet makes data **visible**, so the core skill is learning to look: check stored types before trusting values, match each function and chart to the precise question, surface patterns and anomalies by sorting, filtering, grouping, and pivoting, and find data-quality issues by eye before computing. Answer a business question in three steps — formulate, select operations, interpret — and document every step and judgement so another person can reproduce the work unaided. Look at the data before you touch it.

Exercises

1. Enter a small dataset of at least ten rows, ensuring numbers are stored as numbers and dates as dates, and format it for correct display. Deliberately mis- type one number as text, then find and fix it using the left/right alignment tell.
2. Using **SUM**, **AVERAGE**, **IF**, **COUNTIF**, and one lookup (**VLOOKUP** or **INDEX/MATCH**), answer five simple questions about a dataset. For each, write one sentence justifying why that function — and not a similar one — is the appropriate choice.
3. Sort, filter, and group the same table to surface one clear pattern and one anomaly. Describe in words what each of the three operations revealed.
4. Build a pivot table with one row field, one column field, and a summed measure. Then change the aggregate from sum to average and explain how the business reading of the table changes.
5. Take a deliberately dirty dataset (duplicates, inconsistent country names, some blanks). Identify and correct each issue manually, and keep a running log of every change and the judgement behind it.
6. From appropriate data, construct a bar chart, a line chart, and a scatter plot. For each, state the question it answers and explain why the other two chart types would answer it less well.
7. Find one published or invented chart with a truncated (non-zero) axis. Redraw it with a zero-based axis and describe how the visual impression changes though the data does not.
8. Formulate one genuine business question about your dataset, answer it using only spreadsheet operations, and finish with a single interpretive sentence in the question’s own terms.
9. Write the documentation for the analysis above — source, steps, assumptions, and the cell ranges behind each result — then hand it and the raw data to a classmate and confirm they reproduce your number without asking you anything.